

Cours 06 — GitHub Actions

Module 6/8 — Formation Git & GitHub

Objectifs

À la fin de ce module, vous saurez : - nommer les concepts d'Actions — **workflow**, **événement**, **job**, **step**, **runner**, **action** — et comment ils s'emboîtent ; - écrire un workflow d'**intégration continue** (CI) pour un projet Python : tests et lint à chaque push et chaque PR ; - lire un workflow YAML ligne par ligne et le déclencher/suivre depuis le terminal ; - utiliser une **matrice**, un **secret** et le **cache** de dépendances.

Théorie

L'idée : automatiser ce qui suit chaque commit

GitHub Actions exécute automatiquement des tâches en réponse à des événements de votre dépôt. L'usage le plus fréquent est l'**intégration continue** : à chaque push ou PR, GitHub lance vos tests sur une machine neuve. Si les tests passent, la PR est verte ; s'ils échouent, elle est rouge et — combinée à la protection de branche du cours 04 — **ne peut pas être fusionnée**. Résultat : `main` reste toujours dans un état qui compile et qui passe les tests.

Le vocabulaire, du plus grand au plus petit

Terme	Définition
Workflow	Un fichier YAML dans <code>.github/workflows/</code> , déclenché par des événements
Événement	Ce qui déclenche : <code>push</code> , <code>pull_request</code> , <code>schedule</code> (cron), <code>workflow_dispatch</code> (bouton manuel)...
Job	Un groupe de steps qui tourne sur un runner ; par défaut, les jobs tournent en parallèle
Step	Une étape : soit une commande shell (<code>run:</code>), soit une action réutilisable (<code>uses:</code>)
Runner	La machine virtuelle éphémère qui exécute un job (<code>ubuntu-latest</code> , <code>windows-latest</code> ...)
Action	Une brique réutilisable, publiée par GitHub ou la communauté (<code>actions/checkout@v4</code>)

L'emboîtement : un **workflow** contient des **jobs**, chaque job tourne sur un **runner** et enchaîne des **steps**, dont certains appellent des **actions** toutes faites.

```
workflow (ci.yml)
├─ job « tests »          ← tourne sur un runner ubuntu-latest, neuf à chaque
fois
├─ step: uses checkout    (action : récupère votre code)
├─ step: uses setup-python (action : installe Python)
├─ step: run pip install   (commande shell)
├─ step: run ruff check     (commande shell)
└─ step: run pytest        (commande shell)
```

Le runner est neuf à chaque fois

Point souvent mal compris : le runner est une machine **jetable**. Il naît au début du job, exécute vos steps, puis disparaît. Rien n'est conservé d'une exécution à l'autre — c'est pour ça qu'il faut réinstaller les dépendances à chaque run (et pour ça que le **cache** existe, pour ne pas les **retélécharger** à chaque

fois). Cette fraîcheur est une force : vos tests tournent dans un environnement propre, comme chez un nouveau collègue, ce qui débusque les « ça marche sur ma machine ».

Exemple complet : la CI d'un projet Python

Fichier `.github/workflows/ci.yml` :

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.11", "3.12", "3.13"]

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }
          cache: pip

      - name: Installer les dépendances
        run: |
          pip install -r requirements.txt
          pip install pytest ruff

      - name: Lint
        run: ruff check .

      - name: Tests
        run: pytest -q
```

Décorticage, ligne par ligne

- `name: CI` — le nom affiché dans l'onglet **Actions** du dépôt.
- `on:` — les **déclencheurs**. Ici : chaque push sur `main`, **et** chaque `pull_request` (quelle que soit la branche). C'est la combinaison qui protège `main`.
- `jobs:` → `tests:` — un seul job, nommé `tests`.
- `runs-on: ubuntu-latest` — le runner : une VM Ubuntu neuve.
- `strategy: matrix:` — la **matrice**. `python-version` a trois valeurs, donc GitHub lance **trois jobs en parallèle**, un par version de Python. Un test qui casse sur 3.13 seulement sera visible immédiatement.
- `steps:` — les étapes, dans l'ordre :
 - `uses: actions/checkout@v4` — action officielle qui **récupère votre code** sur le runner (sans elle, le runner est vide) ;

- `uses: actions/setup-python@v5` avec `python-version: ${{ matrix.python-version }}` — installe la version de Python **de la case courante** de la matrice ; `cache: pip` mémorise les dépendances entre exécutions ;
- `run: |` — un bloc de commandes shell (le `|` permet plusieurs lignes) qui installe les dépendances ;
- Lint puis Tests — l'ordre compte : on lance le lint **avant** les tests pour **échouer vite** si le code est mal formé, sans attendre la suite de tests complète.

Commandes commentées

Les workflows se pilotent surtout depuis l'interface web (onglet **Actions**), mais la CLI `gh` (cours 08) permet de tout suivre sans quitter le terminal.

Suivre et déclencher — `gh run / gh workflow`

```
gh run list                # les dernières exécutions de workflow, avec
leur statut
gh run watch               # suivre EN DIRECT l'exécution en cours (barre
de progression)
gh run view --log-failed   # afficher SEULEMENT les logs des steps en échec
gh workflow run ci.yml     # déclencher manuellement un workflow (si
workflow_dispatch)
```

`gh run watch` remplace le rafraîchissement compulsif de la page web. `gh run view --log-failed` va droit au step qui a cassé — inestimable quand un run a vingt steps et qu'un seul est rouge.

Un secret, un déclenchement manuel

```
on:
  workflow_dispatch:      # ajoute un bouton « Run workflow » dans l'onglet
Actions

jobs:
  deploiement:
    runs-on: ubuntu-latest
    steps:
      - name: Déployer
        env:
          API_KEY: ${{ secrets.API_KEY }} # injecté depuis Settings → Secrets and
variables
        run: ./deploy.sh
```

`workflow_dispatch` transforme le workflow en bouton : on le lance à la demande. Un **secret** (défini dans *Settings* → *Secrets and variables* → *Actions*) est chiffré et n'apparaît **jamais** dans les logs — GitHub le masque automatiquement. Ne mettez **jamais** un jeton en clair dans un `.yml` ou un commit : le workflow est public dans un dépôt public.

Exercices pratiques

Reprenez le dépôt du fil conducteur (ajoutez-y un fichier Python et un test si besoin).

1. **Premier workflow.** Ajoutez le workflow CI ci-dessus, avec un test `pytest` minimal (une fonction et son `assert`), poussez, puis vérifiez son exécution verte dans l'onglet **Actions**.
2. **CI rouge.** Ouvrez une PR qui **casse** volontairement le test. Observez le **X** sur la PR, corrigez, repoussez, et constatez le passage au vert.
3. **Matrice élargie.** Modifiez la matrice pour tester aussi sur `windows-latest` (introduisez `matrix.os` et `runs-on: ${{ matrix.os }}`).

4. **Planification.** Créez un **second** workflow déclenché par schedule (cron) qui affiche simplement la date — vérifiez la syntaxe cron dans la documentation GitHub.

Solutions des exercices

Solution 1 — Premier workflow

Un test minimal, `test_exemple.py` à la racine :

```
def additionne(a, b):
    return a + b

def test_additionne():
    assert additionne(2, 3) == 5
```

Créez `.github/workflows/ci.yml` avec le contenu de l'exemple complet ci-dessus, puis :

```
git add .github/workflows/ci.yml test_exemple.py
git commit -m "ci: ajoute l'intégration continue"
git push
```

Onglet **Actions** du dépôt : le workflow « CI » apparaît, avec trois exécutions parallèles (une par version de Python), qui passent au vert. Si `requirements.txt` n'existe pas, retirez la ligne `pip install -r requirements.txt` ou créez un fichier vide.

Solution 2 — CI rouge

```
git switch -c casse-le-test
```

Modifiez le test pour qu'il échoue :

```
def test_additionne():
    assert additionne(2, 3) == 6      # faux exprès
git add . && git commit -m "test: casse volontairement"
git push -u origin casse-le-test
```

Ouvrez la PR : la CI tourne, puis affiche un **X rouge** ; le bouton de fusion est bloqué. Corrigez (remettez `== 5`), committez, poussez sur la même branche :

```
git add . && git commit -m "fix: corrige l'assertion"
git push
```

La CI se relance toute seule et passe au **vert** : la PR redevient fusionnable.

Solution 3 — Matrice élargie

On ajoute une dimension `os` à la matrice et on rend `runs-on` dynamique :

```
jobs:
  tests:
    runs-on: ${ matrix.os }
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        python-version: ["3.11", "3.12", "3.13"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }
      - run: pip install pytest
      - run: pytest -q
```

Deux OS × trois versions = **six** jobs en parallèle. C'est le produit cartésien de la matrice : chaque combinaison est testée isolément.

Solution 4 — Planification (cron)

Second fichier, `.github/workflows/heartbeat.yml` :

```
name: Heartbeat
on:
  schedule:
    - cron: "0 8 * * *"      # tous les jours à 08:00 UTC (min heure jour mois jour-
semaine)
  workflow_dispatch:        # + bouton manuel, pratique pour tester sans attendre

jobs:
  ping:
    runs-on: ubuntu-latest
    steps:
      - run: date
```

Les cinq champs du cron sont minute heure jour-du-mois mois jour-de-semaine. `0 8 * * *` = à la minute 0 de l'heure 8, tous les jours. GitHub exécute les schedule en **UTC** ; ajoutez `workflow_dispatch` pour pouvoir déclencher tout de suite avec `gh workflow run heartbeat.yml`.

Points clés à retenir

- Un workflow = un fichier **YAML** dans `.github/workflows/`, déclenché par des **événements**.
- **CI + protection de branche** = main reste toujours vert : une PR aux tests rouges ne fusionne pas.
- Le runner est **neuf et jetable** à chaque exécution — d'où la réinstallation des dépendances et l'intérêt du cache.
- Une **matrice** teste plusieurs environnements **en parallèle** pour quelques lignes de YAML.
- Les **secrets** passent par `${{ secrets.* }}`, sont masqués dans les logs, et **jamais** en clair dans le code.